Manual: como funciona el sistema de pre-etiquetado PoC-Labelling

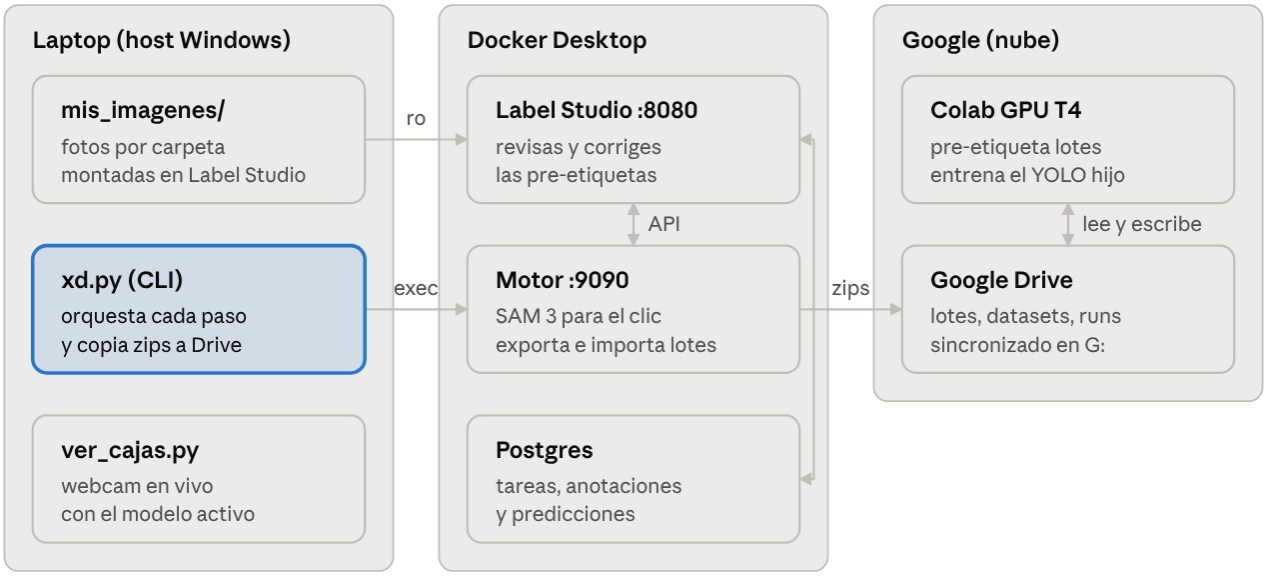
Oct 15, 2026 · @IoT Lab

Este manual explica, paso a paso, que hace cada pieza del sistema que construimos y como funciona por dentro: de describir un caso a tener un modelo propio que pre-etiqueta y cuenta objetos en vivo.

1. Arquitectura

El sistema vive en tres lugares: tu laptop orquesta, Docker guarda y sirve los datos, y Colab pone la GPU que la laptop no tiene.

Tres zonas: tu laptop, Docker y la nube de Google



arquitectura · 3 zonas, 8 piezas

`xd.py` es la puerta de entrada: llama a los scripts del motor con `docker compose exec` y copia a Drive los zips que el motor deja en `motor/lotes`. Label Studio guarda todo en Postgres y habla con el motor por su API REST. Las fotos nunca se duplican: Label Studio las lee de `mis_imagenes/` como carpeta de solo lectura (ro).

2. El ciclo completo

La idea central es un arranque en frio: SAM 3 etiqueta las primeras fotos sin entrenamiento, tu las corriges, y con ellas entrenas un modelo propio que despues pre-etiqueta mejor que SAM 3.

El modelo hijo cierra el ciclo: cada vuelta etiqueta mas con menos trabajo



ciclo de punta a punta · 10 pasos y la vuelta

Los pasos 4 y 6 corren en Colab; el resto, en la laptop. Cada vuelta del ciclo (8 a 5) usa el ultimo modelo activo, asi que revisar cuesta menos cada vez. Las secciones siguientes explican cada paso por dentro.

3. Spec, XML, perfil y version

Todo caso nuevo empieza por una **spec**: un JSON que describe el problema sin escribir codigo. El codigo del motor es general; lo especifico de cada caso vive solo en la spec.

Que contiene una spec: las clases (nombre, prompts en ingles para SAM 3, limites como `area_min`, `area_max`, `max_por_imagen`, color, modo de clic), las carpetas donde se organizan las fotos (tipo pieza, mezcla o negativo), los prompts negativos (por ejemplo "shadow") y la raiz de imagenes.

La cadena de transformaciones

1. `spec.py` valida la spec: rechaza claves desconocidas y avisa de problemas, como dos clases que comparten prompt.

2. `generar.py` convierte la spec en el **XML de Label Studio** de forma determinista: la misma spec siempre produce el mismo XML. Los limites viajan como atributos de cada `<Label>`.
3. `perfil.py` lee ese XML y produce el **perfil**: el diccionario que usa el motor (clases, prompts, umbrales globales como `iou_duplicado` o `contencion`). Si dos clases comparten un prompt, las fusiona con el valor mas permisivo.
4. `version.py` calcula la **version del modelo**: `sam3-<huella del perfil>-<huella del codigo>`.

Por que la version es tan importante: Label Studio guarda cada prediccion con su `model_version` y solo muestra como pre-etiqueta la version que tiene configurada el proyecto. Si cambias el XML o el codigo del nucleo (`reglas.py`, `pipeline.py`), cambia la version y las predicciones viejas quedan huerfanas: Label Studio vuelve a pedirle predicciones al motor. Por eso la logica nueva se puso en modulos aparte y no se toco el nucleo.

Analogia: la version es como el numero de lote de un producto. Si cambias la receta o los ingredientes, el lote es otro, y no mezclas cajas de dos lotes en el mismo estante.

4. Pre-etiquetado con SAM 3 semantico

SAM 3 (Segment Anything Model 3) es un modelo de segmentacion que recibe una imagen y un texto ("yellow plastic part") y devuelve mascaras de todos los objetos que encajan con ese texto, cada una con su confianza. No hay que entrenarlo: por eso sirve para arrancar un proyecto sin ninguna etiqueta.

Que hace `pipeline.inferir()`

1. Junta todos los textos del perfil (prompts de las clases + negativos) y se los pasa a SAM 3 en una sola llamada.
2. Recibe mascaras crudas: decenas por foto, muchas repetidas o basura.
3. Las filtra con `reglas.aplicar()` (abajo).
4. Convierte cada mascara final en un poligono simplificado (`cv2.findContours` + `approxPolyDP`) y lo traduce al formato de Label Studio: coordenadas en porcentaje de la imagen.

Las reglas de post-proceso, en orden (`reglas.py`, funciones puras)

1. **Candidatos**: descarta mascaras con confianza menor que la del prompt o con area fuera de `[area_min, area_max]`.
2. **Sombras**: une las mascaras de los prompts negativos y descarta candidatos cubiertos mas de un 50% por ellas.

3. **Duplicados**: ordena por confianza; si dos mascaras se solapan con IoU > 0.6, se queda la de mayor confianza.
4. **Fragmentos**: si una mascara cae casi entera dentro de otra de la misma clase (umbral `contencion`), es un pedazo y se descarta.
5. **Maximo**: respeta `max_por_imagen`, quedandose con las de mayor confianza.

Su limite (lo que vimos en el diagnostico): si dos clases comparten prompt, SAM 3 no sabe cual es cual. La instancia sale "ambigua" con las dos clases y el codigo se queda con la primera. Asi, Eje_Interno nunca se predecia: todo amarillo salia como Eje_Externo. Este es el motivo por el que existe el modelo hijo.

5. Modo clic interactivo

Cuando revisas una foto en Label Studio y falta una pieza, haces clic sobre ella y el motor devuelve su poligono. Lo resuelve `interactivo.py` con **SAM3Predictor**, la variante de SAM 3 que recibe puntos o cajas en lugar de texto.

Como funciona por dentro

- SAM separa el trabajo en dos partes: el **encoder** convierte la imagen en un mapa de caracteristicas (lento, segundos en CPU) y el **decoder** convierte un clic en una mascara usando ese mapa (rapido).
- Por eso el motor **pre-codifica** las siguientes tareas en segundo plano y guarda sus mapas en una cache. Cuando llegas a una foto, el clic responde casi al instante.
- Un clic puede ser ambiguo (la pieza o el tornillo encima). SAM devuelve varias mascaras candidatas y la clase decide cual usar: `grande`, `chico` o la de mayor confianza (campo `clic` de la spec).
- **Precalentar** (`xd.py precalentar`) carga el modelo antes de empezar: la carga en CPU tarda 80-240 s.

Borrar regiones, cambiar la clase o mover vertices no usa el motor; solo anadir con clic lo necesita.

6. Lotes en Colab: exportar, procesar e importar

La laptop no tiene GPU, asi que el pre-etiquetado masivo se hace en Colab (GPU T4 gratuita). Para no abrir un tunel de red entre Colab y Label Studio, el intercambio es por archivos en Google Drive.

1. **Exportar** (`exportar_lote.py`, en el contenedor): elige las tareas, mete en un zip las fotos, el codigo que hace la inferencia (`codigo/`: reglas, pipeline, hijo, lote) y un `manifiesto.json` con el perfil, la version, el modo y la lista de tareas. En modo hijo

tambien mete `modelo/best.pt`. `xd.py` copia el zip a Drive.

2. **Procesar** (Colab): la celda 2 busca zips sin su `_pred.json`, los extrae e importa el `lote.py` **que viene dentro del zip**. Asi el resultado lo decide el codigo de la version exportada, no el del notebook, y un modo nuevo no obliga a tocar el notebook.
3. **Importar** (`importar_lote.py`): sube cada prediccion con `POST /api/predictions`, salta las tareas que ya tienen esa version, y con `--fijar-version` cambia el `model_version` del proyecto para que Label Studio las muestre.

Idea clave: "el codigo viaja con los datos". Es el mismo principio que un contenedor Docker: el lote lleva todo lo que necesita para reproducir su resultado.

Cuidado: la celda 2 procesa todo zip pendiente. Los lotes que no quieres importar se mueven a `lotes\_archivo`.

7. Muestra estratificada

Al empezar un proyecto no conviene pre-etiquetar ni revisar todo: basta una muestra de 20-50 fotos. El riesgo de elegirlas al azar es que una carpeta pequena (por ejemplo los negativos) quede fuera. La **muestra estratificada** reparte la muestra por carpeta (cada carpeta es un "estrato").

Como reparte `repartir(tamanos, n, minimo)`

1. Da a cada carpeta un minimo (1 por defecto), empezando por las mas grandes, mientras alcance n.
2. Reparte el resto de uno en uno a la carpeta con menor proporcion elegida (cupo / tamano). El resultado queda proporcional al tamano: con carpetas de 100, 50 y 10 fotos y n = 32, salen 20, 10 y 2.
3. Nunca pide mas fotos de las que tiene una carpeta.

Como elige dentro de cada carpeta: ordena por un hash de `semilla:id` y toma las primeras. Es determinista: la misma semilla da siempre la misma muestra, y otra semilla da otra. Con `--prioridad` (por ejemplo los ids que marco el diagnostico), esas tareas entran primero dentro del cupo de su carpeta, sin cambiar los cupos.

De donde sale la carpeta: de la ruta de la tarea en Label Studio, `?d=mis_imagenes/<carpeta>/<archivo>`; `carpeta_de()` toma el directorio padre inmediato.

8. Diagnostico y correccion semiautomatica

Antes de entrenar hay que saber si las etiquetas son buenas: un modelo aprende exactamente lo que le ensenas, errores incluidos.

`diagnostico.py` (solo lectura) compara, por carpeta y por clase, lo anotado con lo predicho y busca anomalias:

- tareas que superan `max_por_imagen` (en ensamble: 74 tareas con 2 Eje_Externo, cuando el maximo es 1);
- regiones con area fuera de los limites de la spec;
- pares de clases que deberian aparecer juntas y no cuadran (`--pares`);
- limites que se pierden al fusionar prompts compartidos.

Tambien muestra la **distribucion de areas** por clase. Ahi se vio que el area no separaba los dos ejes (medianas 0.20% y 0.24% de la imagen), asi que hacia falta otro criterio.

La correccion por forma (`correccion.py`): el eje externo es delgado y largo, el interno ancho. La **elongacion** mide eso:

1. Se pasan los vertices del poligono de porcentajes a pixeles (en porcentajes una imagen 16:9 deforma la forma).
2. Se calcula su matriz de covarianza y sus dos ejes principales (PCA).
3. Elongacion = raiz del cociente entre el eje mayor y el menor: un circulo da 1, una barra de 100 x 10 da 10. No depende de la rotacion.

Si una tarea tiene exactamente 2 regiones del par, la mas alargada pasa a la clase "larga" y la otra a la "ancha". Si la diferencia es menor de 15%, se marca dudosa y no se toca.

Sin escribir en Label Studio: `corregir_pares.py` guarda las correcciones en un JSON y `exportar_yolo.py --correcciones` las aplica en memoria al exportar el dataset. Las anotaciones originales quedan intactas y la correccion se deshace simplemente no pasando el archivo.

9. Dataset YOLO, entrenamiento y KPI

Del Label Studio al formato YOLO (`exportar_yolo.py`)

- Toma la anotacion mas reciente de cada tarea (ignora los Skip y los clics).
- El poligono es la fuente de verdad; la caja se deriva de el. En **det** cada linea del `.txt` es `clase cx cy ancho alto` , normalizado de 0 a 1; en **seg** es `clase x1 y1 x2 y2 ...` .
- El split train/val es **determinista**: un hash md5 del id de la tarea decide si va a val (20%). La misma tarea cae siempre del mismo lado, aunque se agreguen fotos nuevas.
- Se genera un zip con `images/` , `labels/` , `data.yaml` y un `resumen.json` (imagenes, instancias por clase y correcciones aplicadas).

Entrenamiento (notebook en Colab): parte de `yolo11n.pt`, un modelo pequeno preentrenado, y lo ajusta a las clases del proyecto (*fine-tuning*). Con `patience` se detiene si deja de mejorar. Guarda `best.pt`, los pesos de la mejor epoca en validacion.

Por que el KPI es el conteo y no el mAP

- El **mAP50** mide si las cajas predichas coinciden con las reales (IoU >= 0.5), como el area bajo la curva precision-recall de cada clase, promediada entre clases. Es la metrica estandar de deteccion.
- Pero el objetivo del caso es **contar**: saber cuantos tornillos hay en cada imagen. Una imagen solo esta "bien" si todas sus clases tienen el conteo exacto.
- En ensamble, el modelo v2 tuvo mAP50 0.928 y aun asi solo 7 de 33 imagenes (21%) se contaban bien. Un tornillo de mas o de menos no baja mucho el mAP, pero arruina el conteo.
- Por eso el notebook calcula y guarda en `metricas.json` el **conteo exacto por imagen** y el **error absoluto por clase** con `conf = 0.25`.

10. Registro de modelos

Cada proyecto tiene su registro en `motor/modelos/p<N>/registro.json`: la lista de modelos entrenados, cual esta **activo** y el resumen de sus metricas. `xd.py registrar` copia `best.pt` y `metricas.json` desde la corrida de Drive a `motor/modelos/p<N>/<nombre>/` y anade la entrada.

- **Normalizar**: `registro.normalizar()` lee tanto el `metricas.json` viejo (escrito a mano para el v1) como el del notebook nuevo, y los reduce a un resumen comun: dataset, formato, imgsz, conteo exacto, tasa y mAP.
- **Mejor**: `mejor()` elige por tasa de conteo; si empatan, por mAP50, y despues el mas reciente. Un modelo con mas mAP pero peor conteo nunca gana.
- **Activo**: lo eliges tu (`xd.py modelos N --activar X --aplicar`) y puede no ser el "mejor". La tabla marca los dos: `*` activo, `+` mejor por conteo.
- **Por que importa**: el modelo activo es el que usan automaticamente el pre-etiquetado (modo `auto`) y el visor en vivo. Cambiar de modelo no requiere editar la spec ni el codigo.

11. Modelo hijo + SAM 3 por caja

El **modelo hijo** es el YOLO que entrenaste con tus propias etiquetas. Lo combina con SAM 3 para tener lo mejor de los dos: el hijo sabe **que** es cada cosa (aprendio tus clases, incluso las que comparten color) y SAM 3 sabe dibujar **su contorno** con precision.

Como funciona `hijo.inferir_hijo()` por foto

1. El YOLO hijo detecta: devuelve cajas con clase y confianza (>= 0.25).
2. SAM3Predictor codifica la imagen una vez y recibe **todas las cajas en una sola llamada** como prompts. Devuelve una mascara por caja.
3. `elegir_mascara()` decide si la mascara de SAM es razonable para su caja: al menos 50% dentro de la caja ampliada un 10% y que ocupe al menos 15% de la caja. Si no, se usa la caja como poligono. Ademas recorta la mascara a la caja ampliada, para que no se escape a piezas vecinas.
4. `fusionar()` aplica `max_por_imagen` por clase (ya sin fusion de prompts) y produce instancias con **una sola clase**. Los limites de area estan desactivados por defecto, porque el hijo ya se entreno con etiquetas validadas.
5. Cada mascara se convierte a poligono con las mismas funciones del pipeline semantico.

Garantias del sistema

- `validar_clases()` rechaza un hijo cuyas clases no existan en el proyecto.
- La version propia `hijo-<modelo>-<perfil>-<codigo>` incluye `hijo.py` en el hash: cambiar la logica o el modelo produce una version nueva y no se mezclan predicciones.
- Con `--modo auto`, `exportar_lote.py` usa el hijo si el proyecto tiene uno activo.

Resultado medido: en Colab, 0.8 s por foto (contra ~1.8 s del semantico); en 20 tareas, SAM devolvio una mascara por caja en todas y solo 2 regiones cayeron a caja.

12. Tasa de aceptacion

La tasa de aceptacion responde a la pregunta practica: **cuanto trabajo me ahorra el pre-etiquetado**. Compara lo que predijo el modelo con lo que quedo despues de tu revision.

Emparejamiento de regiones (`aceptacion.comparar`), en tres pasadas:

1. **Por id**: al aceptar una prediccion, Label Studio conserva el id de la region.
2. **Por IoU >= 0.5 en la misma clase**, de mayor a menor IoU.
3. **Por IoU >= 0.5 entre clases distintas**: es una region a la que le cambiaste la clase.

Categoria	Significado
aceptada	misma clase e IoU >= 0.9: la dejaste tal cual
geometria	misma clase, pero ajustaste el contorno
clase	le cambiaste la clase

Categoria	Significado
borrada	la predijo el modelo y la quitaste
anadida	la dibujaste tu; el modelo no la vio

Tasa = aceptadas / predichas. La IoU se calcula sobre la caja del poligono, suficiente para medir esfuerzo.

Resultado en la prueba: el hijo tuvo 95% de aceptacion (92 de 97 regiones) frente a 24% de sam3. La comparacion favorece al hijo porque tu revision partio de sus predicciones, y el umbral 0.9 es muy exigente con cajas diminutas como los tornillos; aun asi, la diferencia muestra el valor del ciclo.

13. Prueba en vivo con webcam

`ver_cajas.py` (via `xd.py ver N`) corre en el host, porque Docker en Windows no tiene acceso a la webcam. Toma el modelo activo del registro (o `--modelo X`) y su `imgsz`, y en cada cuadro:

1. lee la imagen de la camara (OpenCV, backend DirectShow en Windows);
2. la pasa por el YOLO hijo con la confianza actual;
3. dibuja las cajas con un color estable por clase (derivado de un hash del nombre) y un panel con el **conteo por clase** y los FPS.

Teclas: `+` / `-` cambian la confianza y, en pausa (`p`), recalculan al instante; `s` guarda la imagen anotada y su conteo en JSON. Con `--imgsz 640` va mas fluido en CPU, a cambio de perder algo de precision en objetos pequenos.

14. Glosario

Termino	Que significa
Pre-etiquetado	Propuesta automatica de etiquetas que una persona solo revisa
Prompt	Texto que describe a SAM 3 que buscar ("blue plastic part")
Mascara	Imagen binaria que marca los pixeles de un objeto
Poligono	Contorno simplificado de una mascara; es lo que guarda Label Studio

Termino	Que significa
IoU	Interseccion sobre union: cuanto se solapan dos regiones (0 a 1)
Confianza	Seguridad del modelo en una deteccion (0 a 1)
Spec	JSON que describe un caso: clases, carpetas, limites
Perfil	Lo que el motor lee del XML de Label Studio para trabajar
model_version	Etiqueta de cada prediccion; Label Studio solo muestra la del proyecto
Lote	Zip con fotos, codigo y manifiesto para procesar en Colab
Muestra estratificada	Muestra repartida por carpeta para que todas esten representadas
Modelo hijo	YOLO entrenado con tus etiquetas, que luego pre-etiqueta
Elongacion	Cociente de ejes principales de una forma: 1 = redondo, alto = alargado
mAP50	Metrica estandar de deteccion con IoU >= 0.5
Conteo exacto	Imagenes donde todas las clases tienen la cantidad correcta (el KPI)
Tasa de aceptacion	Regiones predichas que se aceptaron sin cambios
Fine-tuning	Ajustar un modelo preentrenado a tus clases con pocos datos
Determinista	Misma entrada, mismo resultado siempre (split, muestra, XML)